LISTA DE EXERCÍCIOS

Computação Gráfica com NumPy e Matplotlib

Material de apoio para prática, aprofundamento teórico e desenvolvimento de cenas gráficas em Python.

Informações	Descrição
Tecnologias	Python, NumPy, Matplotlib e Matplotlib.animation
Abrangência	Representação de objetos, propriedades visuais, transformações, composição, câmera, projeções, animação e depuração
Orientação	Implemente, visualize, documente as decisões e registre os casos de erro encontrados

Orientações gerais

- Organize o código em funções reutilizáveis, separando representação, transformação, projeção, desenho e animação.
- Inclua títulos, legendas, eixos e unidades sempre que ajudarem a interpretar a cena.
- Teste valores normais e valores-limite, como escala nula, câmera muito próxima e ângulos extremos.
- Em cada exercício, apresente uma breve análise: o que foi implementado, quais hipóteses foram adotadas e quais limitações permanecem.

1. Representação de objetos bidimensionais

Crie uma cena 2D composta por um triângulo, um retângulo, um círculo e uma estrela.

Requisitos

- Representar os objetos usando vetores e matrizes do NumPy.
- Armazenar separadamente os vértices de cada objeto.
- Desenhar a cena com Matplotlib.
- Utilizar cores, transparência e espessuras de linha diferentes.
- Incluir uma legenda identificando cada objeto.

Conceitos: Representação geométrica, vértices, arestas, faces, cor e transparência.

2. Transformações geométricas em 2D

Implemente funções para aplicar translação, rotação e escala a um objeto bidimensional.

Requisitos

- Utilizar coordenadas homogêneas.
- Criar uma matriz para cada transformação.
- Permitir combinar duas ou mais transformações.
- Apresentar o objeto original e o objeto transformado na mesma figura.
- Comparar a ordem das transformações T · R · S e S · R · T.

Conceitos: Matrizes de transformação, coordenadas homogêneas, composição de transformações e não comutatividade.

$$\mathbf{T \cdot R \cdot S \neq S \cdot R \cdot T \ \ (em\ geral)}$$

3. Construção de uma cena por composição hierárquica

Modele um robô bidimensional utilizando formas geométricas simples.

Requisitos

- Representar cabeça, tronco, braços, mãos, pernas e pés.
- Definir um sistema de coordenadas local para cada parte.
- Posicionar as partes em relação ao corpo principal.
- Aplicar rotações nos braços e nas pernas.
- Permitir alterar a posição e a escala do robô completo.

Conceitos: Composição de objetos, hierarquia, sistemas de coordenadas locais e transformações relativas.

4. Comparação entre projeção ortográfica e perspectiva

Crie uma cena tridimensional formada por cubos posicionados a diferentes distâncias da câmera.

Requisitos

- Representar os vértices dos cubos com NumPy.
- Projetar os pontos usando projeção ortográfica.
- Implementar uma projeção perspectiva simplificada.
- Exibir as duas projeções lado a lado.
- Analisar como a distância da câmera influencia o tamanho aparente.

Conceitos: Coordenadas 3D, câmera, projeção ortográfica, projeção perspectiva e profundidade.

5. Transformações e visualização de um cubo 3D

Implemente a visualização de um cubo tridimensional submetido a diferentes transformações.

Requisitos

- Representar os oito vértices e as seis faces do cubo.
- Aplicar translação, escala e rotações nos eixos X, Y e Z.
- Desenhar as faces com cores diferentes.
- Mostrar o cubo original e o transformado.
- Testar diferentes ordens de rotação.

Conceitos: Modelagem poligonal, transformações 3D, orientação espacial e composição de matrizes.

6. Câmera orbital em uma cena tridimensional

Crie uma cena com vários objetos e uma câmera que orbita ao redor do centro da cena.

Requisitos

- Posicionar pelo menos três objetos em coordenadas diferentes.
- Calcular a posição da câmera com coordenadas polares ou esféricas.
- Variar os ângulos azimutal e de elevação.
- Gerar vários quadros para representar o movimento da câmera.
- Salvar ou exibir a sequência de posições da câmera.

Conceitos: Manipulação de câmera, órbita, ponto de observação, vetor de direção e projeção.

7. Modelo geométrico de um autódromo

Modele um autódromo simplificado com uma pista e veículos representados por formas geométricas.

Requisitos

- Construir a pista com segmentos de reta e curvas.
- Representar cada carro por um retângulo ou conjunto de polígonos.
- Definir uma trajetória para os veículos.
- Calcular a posição dos carros ao longo da pista.
- Orientar os veículos de acordo com a direção do movimento.
- Comparar diferentes velocidades.

Conceitos: Composição de cenas, trajetórias, translação, rotação, orientação e instanciamento de objetos.

8. Animação baseada em Δt

Crie uma animação de objetos se movimentando com velocidade independente da taxa de atualização.

Requisitos

- Usar posição_nova = posição_atual + velocidade · Δt.
- Criar pelo menos três objetos com velocidades diferentes.
- Implementar movimento contínuo.
- Fazer os objetos ricochetearem nas bordas.
- Comparar passos fixos com a atualização usando Δt.
- Utilizar matplotlib.animation.FuncAnimation.

Conceitos: Animação, tempo entre quadros, velocidade, atualização de estados e independência da taxa de quadros.

$$\text{posição_nova} = \text{posição_atual} + \text{velocidade} \cdot \Delta t$$

9. Animação hierárquica de um corpo humano simplificado

Construa uma representação geométrica simplificada de um corpo humano e anime seus movimentos.

Requisitos

- Representar o corpo com segmentos de reta, círculos e elipses.
- Organizar o modelo em tronco, cabeça, braços e pernas.
- Animar caminhada ou agachamento.
- Aplicar rotações coordenadas às articulações.
- Usar funções seno e cosseno para controlar os movimentos.
- Incluir opções para pausar e reiniciar.

Conceitos: Hierarquia de objetos, articulações, rotação em torno de pivôs, animação procedural e sincronização.

10. Diagnóstico de erros em uma cena gráfica

Desenvolva uma cena 2D ou 3D e inclua intencionalmente problemas comuns para identificá-los e corrigi-los.

Requisitos

- Apresentar a cena com pelo menos três erros.
- Explicar a causa de cada problema.
- Corrigir os erros e comparar antes/depois.
- Criar mensagens de validação para entradas inválidas.
- Registrar as hipóteses e os testes usados no diagnóstico.

Conceitos: Depuração gráfica, validação de dados, projeção, sistemas de coordenadas, transformações e exceções.